Contents

How This Project Was Built — Step-by-Step Walkthrough **1**
0. The goal (what the assignment asked) . . . 1
1. Architecture decisions (the "why") . . . 2
2. The pipeline, stage by stage . . . 2
   Stage A — Configuration & manifest (`config.py`, `manifest.py`) . . . 2
   Stage B — Get the signals (ingestion) . . . 3
   Stage C — Segmentation (`ingest_real.py: segment_signal`) . . . 3
   Stage D — Scalograms (`scalogram.py`) . . . 3
   Stage E — Leakage-free split (`split.py`) . . . 3
   Stage F — Class balancing (`balance.py`) . . . 4
   Stage G — Data loader (`data_loader.py`) . . . 4
   Stage H — Models (`model.py`) . . . 4
   Stage I — Training (`train.py`) . . . 4
   Stage J — Evaluation (`evaluate.py`) . . . 4
   Stage K — Fusion training (`train_fusion.py`) . . . 4
   Stage L — Ablation experiments (`experiments.py`) . . . 5
3. The results we obtained (real KAIST data, held-out recordings) . . . 5
4. Quality, reproducibility, packaging . . . 5
5. Deliverables produced (`docs/build/`) . . . 6
6. Environment lessons (so you can re-run it) . . . 6
7. The exact order things were built (chronological) . . . 6

How This Project Was Built — Step-by-Step Walkthrough

A complete, honest account of *what* was built, *why* each decision was made, and *how* every piece fits together. Read this with `README.md` (quickstart) and `docs/report/report.md` (the formal report). This document is the "engineering diary".

> **Toolchain in one sentence:** the project runs **end-to-end in Python only** (PyWavelets for the wavelet transform, TensorFlow/Keras for the CNN). The `matlab/` scripts are an **optional alternative**, not a dependency — every reported result was produced by Python.

---

0. The goal (what the assignment asked)

Build a model that: 1. takes **PMSM motor signals** (current and/or vibration), 2. converts short signal windows into **Wavelet Scalogram images** (via the Continuous Wavelet Transform, CWT), 3. classifies those images with a **Convolutional Neural Network (CNN)** to detect and identify faults (healthy vs inter-turn short, plus demagnetization / overload as extra classes).

Deliverables: a labelled image dataset, training code, a final CNN model, result figures, a 25–35 page report, and a 15–20 slide presentation.

---

1. Architecture decisions (the “why”)

Before writing code, these choices were locked in:

Decision	Choice	Why
Language	**Python** (MATLAB optional)	Free, reproducible, no toolbox licences; TensorFlow + PyWavelets cover everything.
Signal sources	**Real data + synthetic + (optional) MATLAB sim**	Real data for credibility; synthetic for instant validation and the missing classes.
Channels	**Current AND vibration**	Compare which sensor detects faults better — a real research question.
Classes	**4** (Healthy, InterTurn, Demagnetization, Overload)	Matches the assignment; real data covers 2, synthetic covers the other 2.
Single source of truth	**`data/manifest.csv`**	One table links every signal → scalogram → label → split. No hidden state.
Config	**`config.yaml`**	Every parameter in one file; modules never hard-code values.
Method	**TDD** (write tests first)	Catch bugs early; prove correctness for a research artefact.

The full design rationale is in `docs/superpowers/specs/` and the step plan in `docs/superpowers/plans/`.

---

2. The pipeline, stage by stage

The data flows: **raw signal → segment → CWT scalogram → CNN → fault class**. Each stage is one Python module under `python/`.

Stage A — Configuration & manifest (`config.py`, `manifest.py`)

- `config.py` loads `config.yaml` and validates required keys (raises if missing).
- `manifest.py` defines the columns `signal_id, source, signal_type, class, severity, fs, dataset_name, recording_id, split, scalogram_path` and the CRUD helpers (`new_manifest`, `add_record`, `save/load_manifest`). Every later stage reads/writes this one CSV.

Stage B — Get the signals (ingestion)

Three interchangeable sources, each writing rows into the manifest:

1. **Real KAIST dataset** (`ingest_mendeley.py`) — the primary source.
   - Dataset: *Vibration & Current Dataset of 3-phase PMSM with Stator Faults* (Mendeley `rgn5brrgrn`, CC-BY-4.0). Current @ **100 kHz**, vibration @ **25.6 kHz**, stored as **TDMS** files; conditions: normal, inter-turn, inter-coil at several severities.
   - The loader parses each filename (`1000W_0_00_current_interturn.tdms` → power, severity, modality, fault), reads the TDMS via `npTDMS`, **decimates** to 10 kHz (anti-aliased FIR), **segments** into 0.5 s windows with 50 % overlap, caps **50 segments per recording** (even stride so they span the whole run), and writes `.npy` segments + manifest rows.
   - Label map: `0 %` severity → `Healthy`; `inter-turn`/`inter-coil` → `InterTurn`.
2. **Synthetic generator** (`simulate.py`) — builds current signals for all 4 classes with physically-motivated MCSA signatures (harmonics, side-bands, sub-harmonics). Gives instant end-to-end validation and the only Demagnetization/Overload examples.
3. **MATLAB simulation** (`matlab/`) — optional FOC + Simscape signal export.

> **Note on the data audit:** a second public dataset (Zenodo 13974503) was evaluated and **rejected** because it is 10 Hz tabular data — far too slow for a wavelet transform. This is documented in `docs/data-audit.md`. Knowing *why a dataset was rejected* is a likely defense question.

Stage C — Segmentation (`ingest_real.py: segment_signal`)

Pure function: slices a 1-D array into overlapping windows. Unit-tested independently of any data so the windowing math is provably correct.

Stage D — Scalograms (`scalogram.py`)

- `compute_scalogram()` applies the **complex Morlet** wavelet (`cmor1.5-1.0`) over **128 scales** (geometric spacing 4→256) using `pywt.cwt`, returning the magnitude `|CWT|`.
- `save_scalogram_png()` normalises, maps through the `jet` colormap, and saves a **224×224 RGB PNG** under `data/scalograms/<channel>/<class>/`.
- `generate_scalograms()` renders every manifest row and is **resumable** (skips already-rendered images). The real dataset produced **3,150 scalograms**.

Stage E — Leakage-free split (`split.py`)

- `assign_splits()` groups by **`recording_id`** and assigns whole recordings to train/val/test (70/15/15), stratified by class, **independently per channel**.
- **Why grouping matters:** adjacent windows of one recording are nearly identical. A random split would put near-duplicates in both train and test → **data leakage** → fake-high scores. Grouping by recording prevents this.
- A subtle fix: with only **4 healthy recordings**, naive rounding sent all of them to train. The function was changed to **guarantee ≥1 recording per split** for any class that has enough recordings. Covered by tests.

Stage F — Class balancing (`balance.py`)

- The real data is wildly imbalanced (~4 healthy vs ~60 fault recordings). Trained raw, the CNN just predicts "fault" for everything → 80 % accuracy but **0 % healthy detection** (majority-class collapse).
- `balance_df()` **undersamples the majority class in train/val only**; the **test set stays at its natural distribution** so metrics reflect reality.
- This is why we report **balanced accuracy** and **macro-F1**, not raw accuracy.

Stage G — Data loader (`data_loader.py`)

- `class_to_index()` (pure, tested) maps class names → integer labels.
- `make_dataset()` builds a `tf.data.Dataset` that reads PNGs, resizes to `image_size`, scales to [0,1], shuffles (train), and optionally augments (random horizontal flip). TensorFlow is imported lazily so the pure helpers stay testable without TF installed.

Stage H — Models (`model.py`)

- `build_cnn()` — the baseline single-channel CNN: three Conv→ReLU→MaxPool blocks (32, 64, 128 filters) → Flatten → Dropout(0.5) → Dense(128) → Softmax.
- `build_fusion_cnn()` — dual-branch: one conv branch per channel (current + vibration), each ending in **Global Average Pooling**, concatenated, then a shared dense head. (GAP, not Flatten, because flattening two 26×26×128 maps makes a huge dense layer that ran the machine out of memory.)

Stage I — Training (`train.py`)

- `train_from_df()` builds train/val datasets, optionally computes inverse-frequency **class weights**, compiles with Adam + sparse categorical cross-entropy, and fits with **EarlyStopping(patience=5, restore_best_weights)**.
- `train()` wires in `config.yaml`, applies balancing, saves the `.keras` model and the training history JSON.

Stage J — Evaluation (`evaluate.py`)

- `metrics_from_predictions()` (pure, tested) computes the confusion matrix and classification report; it passes `labels=` so it works even when only 2 of 4 classes appear in the real test set (a bug that was found and fixed).
- `main()` loads the model, predicts on the **test** split, saves the confusion matrix PNG and the per-class report JSON.

Stage K — Fusion training (`train_fusion.py`)

- `pair_manifest()` pairs each current scalogram with the vibration scalogram of the **same condition and segment index**, and splits at **condition level** so both modalities of a condition land in the same split (no leakage).
- Trains/evaluates the dual-branch model; saves `cnn_fusion.keras` and its report.

Stage L — Ablation experiments (`experiments.py`)

- Runs three studies on the real data: (1) balancing on/off, (2) image size 96/160/224, (3) learning curve at 25/50/100 % of the training set.
- **Engineering detail:** each run executes in its **own subprocess** (so TensorFlow frees memory between runs — running 16 trainings in one process exhausted RAM), and results are **checkpointed after every run** (so a crash or laptop-sleep never loses completed work; the sweep resumes from the checkpoint).
- Writes `results/experiments_real.json` and `results/learning_curve.png`.

---

3. The results we obtained (real KAIST data, held-out recordings)

Headline (Healthy vs Inter-turn, balanced training, natural test set):

Channel	Balanced accuracy	Macro-F1
Vibration	**1.00**	**1.00**
Fusion (current+vibration)	0.88	0.76
Current	0.69	0.49

Ablation findings: - **Balancing** drives healthy recall from ~0 → 1.00 on current (kills collapse); no effect on vibration (already perfect). - **Image size** helps current monotonically (0.68 → 0.76 for 96 → 224 px); vibration is saturated at every size (so 96 px would be ~5× faster, free). - **Learning curve** — vibration reaches 0.97 with only ~46 images; current is flat at ~0.70 no matter the data volume → its ceiling is **signal quality, not quantity**.

One honest limitation: only **4 distinct healthy recordings** exist, so the perfect vibration score can't fully exclude the model keying on recording-specific traits. This is stated everywhere (report, slides, README).

---

4. Quality, reproducibility, packaging

- **38 unit tests** (`python/tests/`) cover config, manifest, split (incl. the small-class guarantee), segmentation, KAIST filename parsing, balancing, class weights, scalogram shape, the synthetic generator, data-loader indexing, both models, fusion pairing, a training smoke test, and evaluation metrics. TF-needing tests skip cleanly when TF is absent.
- **Continuous integration** (`.github/workflows/ci.yml`): runs the test suite on Python 3.10/3.11/3.12 on every push. Badge in the README.
- **`Makefile`** targets: `setup, test, demo, simulate, scalograms, split, train, evaluate, train-fusion, report, docs, docs-ar, clean`.
- **`CITATION.cff`**, **MIT `LICENSE`**, comprehensive **`README.md`**.
- **GPU**: the local NVIDIA Quadro P620 was enabled (`pip install 'tensorflow[and-cuda]'`); training runs ~17× faster per step than CPU. Results are identical CPU vs GPU (speed only).

---

5. Deliverables produced (`docs/build/`)

Type	English	Arabic
Report	`report.pdf` (29 pp), `report.docx`	`report-ar.pdf`, `report-ar.docx`
Slides	`slides.pptx`, `slides.pdf`	`slides-ar.pptx`, `slides-ar.pdf`

Source Markdown lives in `docs/report/` and `docs/presentation/`; rebuild with `make docs` (English) and `make docs-ar` (Arabic, RTL, Amiri font).

---

6. Environment lessons (so you can re-run it)

- Use the **Python 3.10 venv** (`.venv`); the machine's default `python3` is 3.14 which has no TensorFlow wheel.
- The dev laptop **suspends when idle and runs on battery**, which killed long background jobs twice. Fix: run long jobs under `systemd-inhibit --what=sleep ...` and keep it plugged in. `experiments.py` also checkpoints/resumes so nothing is lost.
- `pip` hangs on the large `nvidia_cublas_cu12` wheel — fetch that one wheel with `curl -L -C -` then `pip install` it, after which pip finishes the rest.

---

7. The exact order things were built (chronological)

1. Repo analysis + `CLAUDE.md`; design spec + implementation plan.
2. Core Python skeleton (config, manifest, split) with tests — **TDD**.
3. Scalogram module (PyWavelets) + synthetic generator → `make demo` works.
4. Data audit → chose KAIST, rejected Zenodo.
5. KAIST TDMS loader; user downloaded the dataset; ingested 3,150 segments.
6. Rendered scalograms; fixed the split for the 4-healthy-recording case.
7. Trained current + vibration; found the majority-class collapse; added balancing + fixed the evaluation bug.
8. Built + trained the fusion model (added GAP to fix OOM).
9. Ablation experiments (subprocess-isolated, checkpointed) → `experiments.md`.
10. Wrote the full report + slides; built PDF/DOCX/PPTX.
11. Added CI, CITATION.cff; enabled the GPU.
12. Produced the Arabic report + slides and bilingual cover pages.

Everything is committed and pushed to `github.com/molhamfetnah/pmsm-fault-diagnosis-cnn-scalogram`.